



Projet Informatique Individuel *Neuron*²

Rapport de projet

27/01/2025

Table des matières

Introduction	1
Remise en contexte	1
Les données	2
Justifications techniques	2
Modèles d'entraînement	3
KMeans	3
Méthodes d'initialisation de barycentre testés	4
Application Unity	5
Structure de l'application	5
Navigation	5
Game Objects	6
Scripts	7
Quelques captures d'écrans de l'éditeur Unity	8
Onglet : À Propos	9
Onglet : Paramètres	9
Fenêtre de test d'entraînement du MLP	10
Entraînement du MLP en cours	10
Planning final	11
Difficultés rencontrées	11
Les données	11
Unity	11
Exportation du projet	11
Perceptron multi-couches	12
Post Mortem	12

Introduction

*Neuron*² a été un projet dont le sujet a évolué tout le long des 4 mois qui se sont écoulés depuis le lancement des P2I. Initialement prévu pour développer des algorithmes de machine learning appliqué à des données médicales (EEG), son but est devenu de plus en plus pédagogique au cours du temps.

Désormais, le livrable final est un logiciel Unity permettant aux utilisateurs de jouer avec 2 types de modèles d'apprentissage : K Moyennes et un Perceptron multi-couches à descente de gradient.

L'objectif initial était de construire les modèles "à la main", c'est-à-dire sans utiliser des librairies de code.

Cela a été réussi pour KMeans.

Le réseau de neurones, au final, malgré nombreux efforts, a été construit en utilisant un Framework.

Remise en contexte

Pour expliquer le projet, on peut le diviser en deux parties principales. Une première partie sur les données, et l'entraînement des modèles à classifier les données. Une deuxième partie sur la création du logiciel Unity.

Les données

Le jeu de données qu'on a choisi de classifier avec nos modèles d'apprentissage est MNIST. Ce jeu de données est composé de 60'000 images de 28*28 pixels. Chacun de ses pixels à une valeur comprise entre 0 et 255, correspondant à un coloris de gris (0 étant le noir et 255 étant blanc). Chaque image correspond à un "point" à classifier. Les images représentent des chiffres manuscrits, allant du 0 jusqu'à 9. Pour modéliser ce problème dans le code, chaque image, ou matrice d'entiers, a été transformée en un simple vecteur de taille 784, ou chaque 28 valeurs correspondent à une ligne de pixels de l'image. Chaque point possède aussi un label, qui correspond au chiffre que l'image représente vraiment.

Voici un exemple avec quelques images du jeu de données :



Justifications techniques

Ayant déjà manipulé certaines bibliothèques de machine learning en python, notamment PyTorch, j'ai décidé d'utiliser cette fois-ci C#, pour apprendre à appliquer une langue de programmation différente à l'intelligence artificielle, et pouvoir aussi intégrer l'algorithme à l'application Unity plus facilement.

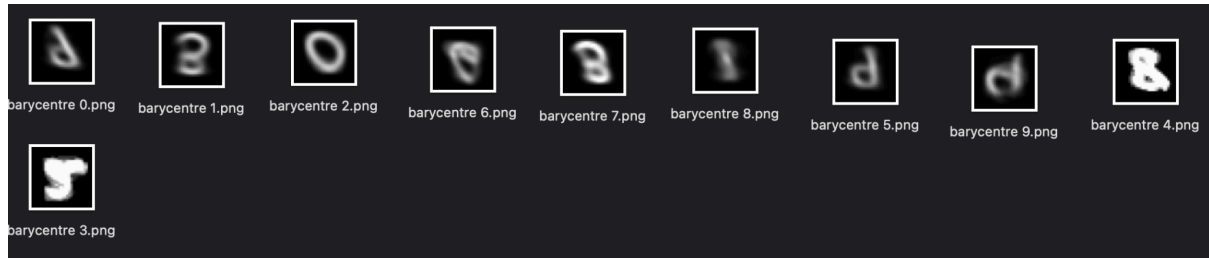
Unity à été choisi comme moteur de développement, d'une part, par facilité de développement sur MacOS, d'autre part par sa compatibilité avec c#, et dernièrement à cause d'une envie personnelle d'acquérir des compétences dans ce domaine. Unity aurait aussi permis l'ajout simplifié d'animations et d'effets sonores. Malheureusement par manque de temps, ses caractéristiques n'ont pas pu être ajoutées à temps au projet.

Construire les modèles de machine learning à la main a ses avantages quand on désire l'intégrer à Unity. Cela permet de pouvoir manipuler l'algorithme et les données avec plus de liberté et jouer plus facilement avec les divers paramètres du modèle.

Afin de pouvoir tester en amont les modèles, et les intégrer à Unity qu'une fois fonctionnels, on admet un projet .NET avec quelques classes, appelé 'Unsupervised_Learning'. Il porte très mal son nom, vu que le perceptron multi-couches repose sur la descente de gradient, qui est une méthode d'apprentissage supervisé, qui vise à minimiser la fonction Cross Entropy. Il s'appelle ainsi parce qu'il

a été à un moment où le projet n'envisageait qu' à utiliser KMeans à des données EEG du cerveau (cf. rendu intermédiaire).

L'intégration de KMeans à Unity s'est déroulée très bien. La classe Simulation 2 s'est reconvertie en KMeansSimulator, et elle permet de simuler plusieurs fois l'algorithme de KMeans en s'appuyant sur des classes Number. Les résultats étaient plutôt positifs! Quand on affiche les résultats obtenus, on obtient des barycentres qui ressemblent plus au moins à des chiffres :



Barycentres obtenus après quelques itérations de KMeans

L'idée était la même pour le perceptron multi-couches, et c'est ainsi que la classe DeepLearner était créée. Puis beaucoup de temps a été dépensé à essayer de la faire marcher. Sans succès.

Finalement, afin d'éviter de perdre plus de temps sur cette aventure, un Framework d'IA natif a .NET et C# à été utilisé pour créer le perceptron multi-couches. Ce Framework était Encog Framework.

D'autres frameworks plus connus ont été testés, notamment TensorFlow et Keras. Mais Encog les a tous battus en facilité de prise en main.

Modèles d'entraînement

KMeans

KMeans est un algorithme d'apprentissage non supervisé qui vise à clusteriser les données d'entraînement. Il a été choisi quand le sujet de projet était encore de classifier des données EEG afin d'essayer de comparer ses résultats avec la classification que nous comme humains nous faisons de ses données. Pour résumer la problématique, on avait un jeu de données extrait du site OpenNeuro, avec 88 enregistrements de signaux EEG. Le jeu de données était composé de 3 classes :

- Des enregistrements EEG de personnes atteintes d'alzheimer
- Des enregistrements EEG de personnes atteintes de démence pré frontale.
- Des enregistrements EEG de personnes sans conditions de santé particulières.

Le but étant de voir si KMeans était capable de faire la même distinction entre les 3 classes.

Finalement, l'algorithme a été appliqué au jeu de données MNIST.

Le principe de KMeans est simple. On initialise des barycentres selon une certaine méthode, on choisit le nombre de clusters (K) à créer. Pour chaque itération d'apprentissage, tous les points de données sont assignés un barycentre d'appartenance en calculant la distance à tous les barycentres

et en choisissant le plus proche. Ensuite, chaque barycentre est mis à jour en le plaçant au “milieu spatiale”, au barycentre (dont le nom) des points qui lui appartiennent. Le calcul de distance peut aussi être choisi de façon arbitraire. En l'occurrence, la distance euclidienne est celle qui a le mieux marché pour nos propos.

Méthodes d'initialisation de barycentre testés

Plusieurs méthodes d'initialisation de barycentre ont été testées. Elles sont toutes disponibles sur l'application sous présentées comme :

- Créer aléatoirement le barycentre
- Choisir aléatoirement le barycentre parmi les données
- Smart KMeans, choisir des barycentre afin que tous aient un label différent

Chacune de ses méthodes à ses avantages et ses inconvénients. Aucune des 3 n'a vraiment réussi à faire une vraie classification de données. Pourtant les résultats restent intéressants à analyser. Par exemple, les 1 finissent souvent dans la même classe que les 7. Les 5, les 8 et les 3 ont une tendance à se rapprocher, tandis que le 2 et le 9 se font remarquer très vite.

Comme observé dans l'image en haut de la page, les résultats ne sont pas très efficaces même en changeant la méthode d'initialisation de barycentres. C'est à ce moment du projet que l'idée de faire un MLP (multilayer perceptron) est venue pour la première fois.

Perceptron multi couches

Afin de réaliser le modèle à la main, ce qui était le but au début, plusieurs variables ont dû être créés:

- H : le nombre de neurones par couche cachée
- L : le nombre de couches cachées
- Z = new double[H,L] : une matrice de double avec les neurones des couches cachées avant activation
- A = new double[H,L] : une matrice de double avec les neurones des couches cachées cachées après activation
- List<double[,]> W : une liste de matrices de poids :
- List<double[]> B : une liste de vecteurs de biais :

Les fonctions, disponibles dans la classe DeepLearner du projet Unsupervised_Learning, ont toutes été faites afin de simuler la propagation forward pour prédire la classe d'une image. Backpropagation visait à corriger les différents poids et biais afin d'améliorer la précision du modèle après chaque epoch. À chaque fois que l'algorithme était testé, la Cross Entropy atteignait un seuil. Plusieurs changements ont été testés. Les fonctions d'activations sigmoïde, tanh et ReLU ont été testées. A chaque fois on devait normaliser l'image de façon différente afin que les valeurs entre 0 et 255 soient entre 0 et 1 pour la sigmoïde, entre -1 et 1 pour Tanh et ReLU. La précision n'a jamais dépassé 13% n'importe les changements proposés.

C'est pour cela que finalement, le réseau a été construit en utilisant le framework Encog :

```

// Reference
public void CreateNetwork(int H, int L)
{
    var network = new BasicNetwork();
    network.AddLayer(new BasicLayer(null, true, ImgSize));

    for (int i = 0; i < L; i++)
        network.AddLayer(new BasicLayer(new ActivationSigmoid(), true, H));

    network.AddLayer(new BasicLayer(new ActivationSoftMax(), false, OutputSize));

    network.Structure.FinalizeStructure();
    network.Reset();
    Network = network;
}

```

Plusieurs configurations ont été testées au début, mais celle qui marchait le mieux était d'utiliser la sigmoïde comme fonction d'activation pour les couches cachées.

Application Unity

Le concept général du projet n'ayant pas changé depuis le début, une grande partie du travail est toujours de faire une application Unity qui permette à l'utilisateur de lancer, et tester par lui-même l'efficacité (ou non) de l'algorithme de K-Means.

L'application Unity possède déjà plusieurs "scènes", boutons, et hyperliens qui permettent à l'utilisateur de naviguer entre les plusieurs fenêtres. L'idée est de faire une application téléchargeable par tout le monde, qui permette à l'utilisateur de jouer avec plusieurs paramètres, plusieurs types d'algorithmes de clustering, et de visualiser leur effet sur la classification finale.

Structure de l'application

Navigation

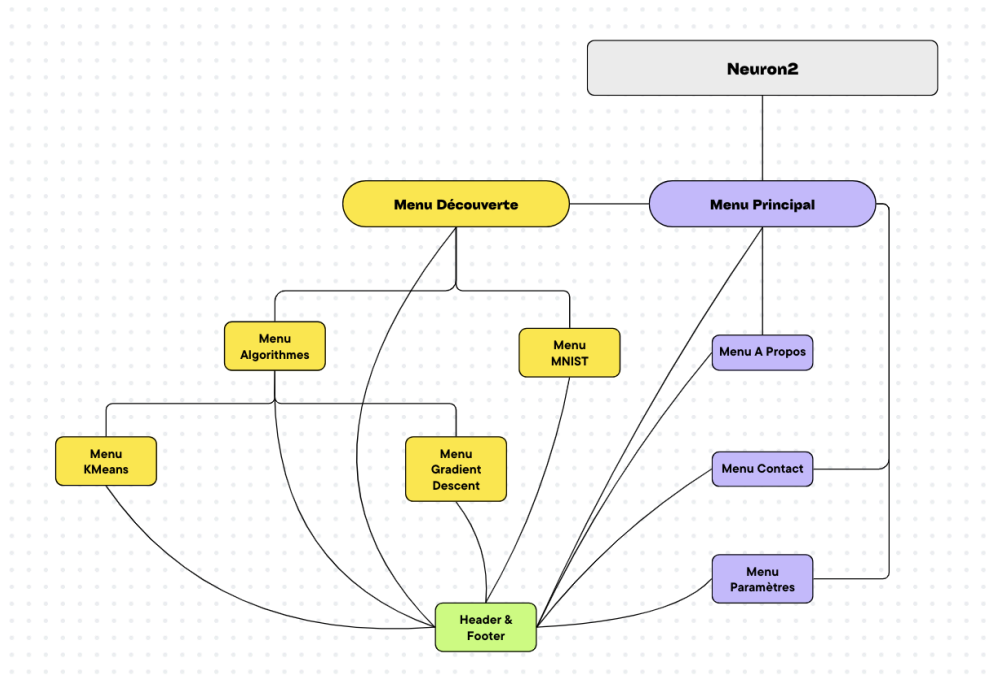
L'application Unity est composée de 2 scènes : Menu Principal et Menu Découverte

Le but de la scène 'Menu Principal' est de présenter un peu le logiciel et l'auteur.

Tandis que la scène 'Menu Découverte' est là où tous les apprentissages ont lieu.

Chaque scène est composée de plusieurs sous-menus, et la navigation entre chaque sous-menu est possible en cliquant sur les différents boutons affichés dans la page, dans le header, et le footer.

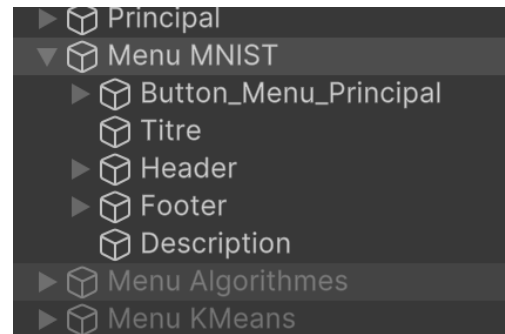
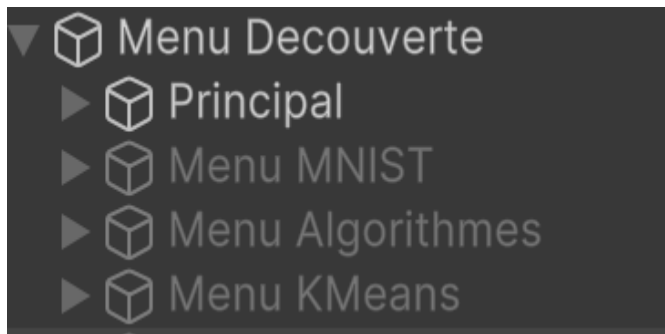
Ainsi, c'est possible de naviguer très rapidement entre scènes grâce aux header et footer :



Game Objects

La construction de tous les éléments visibles sur Unity se fait à travers des Gameobjects, et chaque Gameobject peut incorporer plusieurs composants. Dans le cas de mon projet, ses composants se limitent à des images, des objets UI (d'interface, ex : boutons, sliders, formulaires de texte, etc...), des textures, et des scripts c#. Les scripts sont des classes c# où l'on peut écrire du code, et ils nous permettent de faire communiquer les objets entre eux.

En plus des composants, les Gameobjects peuvent aussi avoir des 'enfants'. Les enfants sont d'autres Gameobjects, et on les relie à leur père, leur position, et pleins d'autres caractéristiques peuvent être activés/désactivés juste en les modifiant dans les père. Cette hiérarchie est visible à tout moment dans l'éditeur Unity.

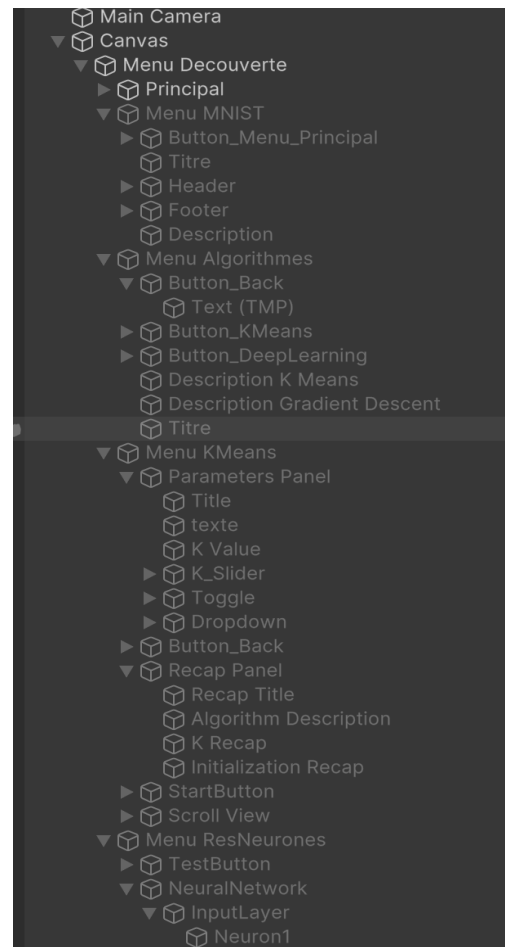


Par exemple, on peut observer l'objet vide Menu Decouverte, qui possède plusieurs enfants : Menu MNIST, Menu Algorithmes, Menu KMeans....

Chacun de ses menus possède aussi des fils, comme peuvent être des boutons, des champs de textes, etc...

La navigation est donc possible en désactivant et en activant ses composants. Par exemple, si on va du Menu Principal au Menu Algorithmes, on a qu'à désactiver le Gameobject Principal, et à activer le Gameobject Algorithmes.

Un schéma complet avec tous les Gameobjects serait d'une taille considérable, on laisserai donc une image illustrer la hiérarchie de la scène menu découverte :



Scripts

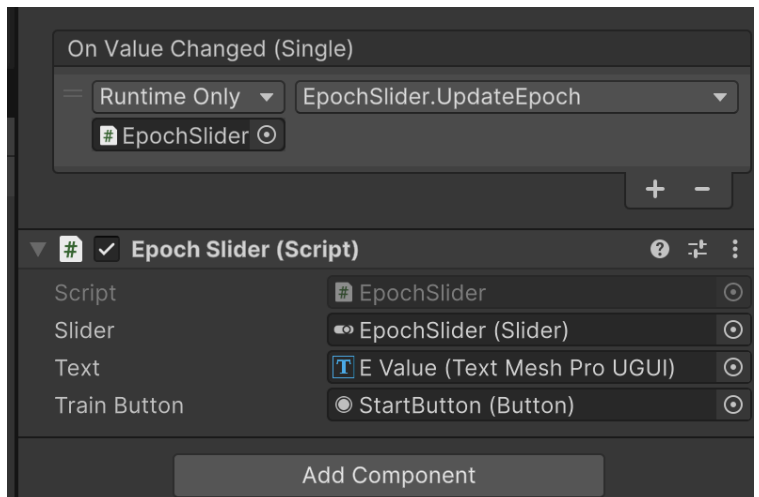
Un code c# est gardé sous forme de script. Ses scripts ont été écrits avec Visual Studio Code et gardés dans des classes. On a les classes normales qu'on peut utiliser même en dehors de Unity, et on peut aussi utiliser la classe MonoBehaviour. Tous les scripts générés par défaut par Unity vont hériter de MonoBehaviour, et c'est cette classe qui nous permet d'interagir avec les Gameobjects de la hiérarchie. Elle nous permet de référencer des objets à l'intérieur des scripts et de changer leurs propriétés à travers du code.

Exemple simple d'un script appartenant au slider d'epochs dans le menu de descente de gradient :


```

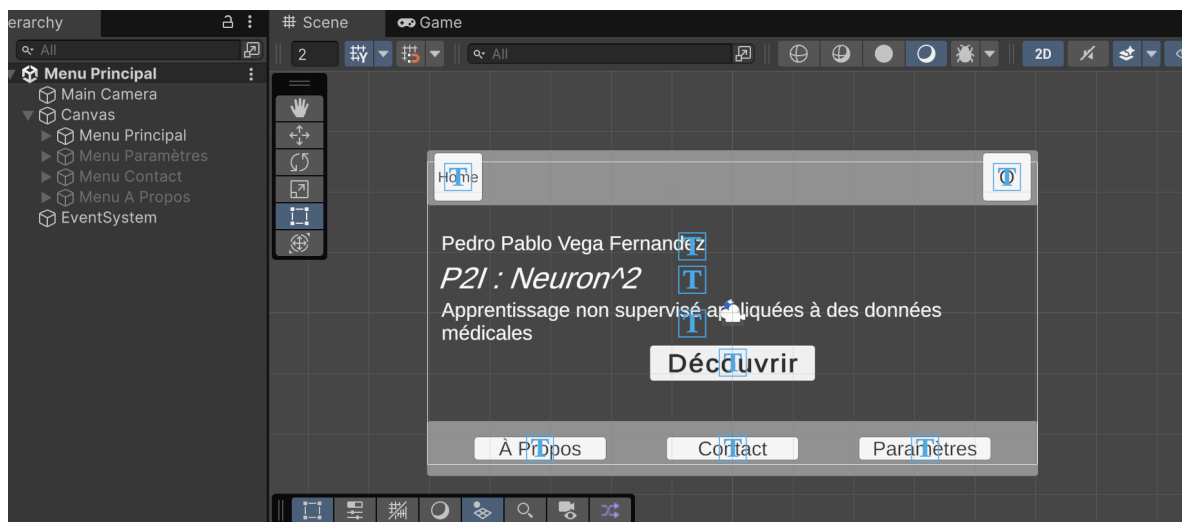
0 references
public class EpochSlider : MonoBehaviour
{
    // Start is called once before the first execution of Upd
    1 reference
    public Slider slider;
    1 reference
    public TMP_Text text;
    2 references
    public Button TrainButton;
    0 references
    void Start()
    {
        0 references
        public void UpdateEpoch()
        {
            float val = slider.value;
            text.text = $"Number of training pochs = {val:F0}";
            if (!TrainButton.enabled)
            {
                TrainButton.enabled = true;
            }
        }
    }
}

```

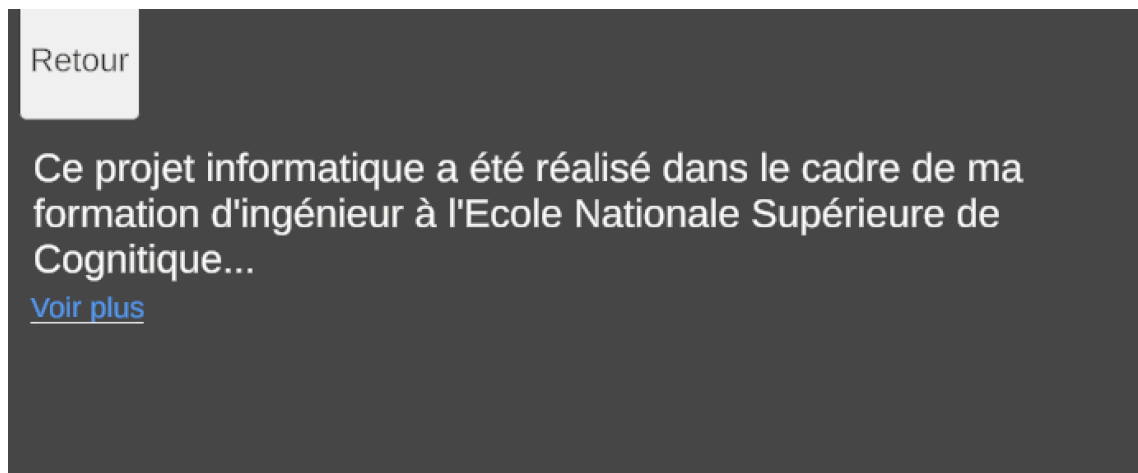


Dans ce cas, la classe EpochSlider hérite de la classe MonoBehaviour, ce qui lui permet d'avoir comme variable de classe un slider, une zone de texte, et un bouton dont on peut accéder et modifier les caractéristiques à travers le code. À travers l'inspecteur de l'objet EpochSlider, on peut faire en sorte que à chaque fois que le slider change de valeur (On Value Changed), la méthode UpdateEpoch() est appelé, et ainsi de suite le texte E Value sera mis à jour pour afficher la valeur actuelle du slider.

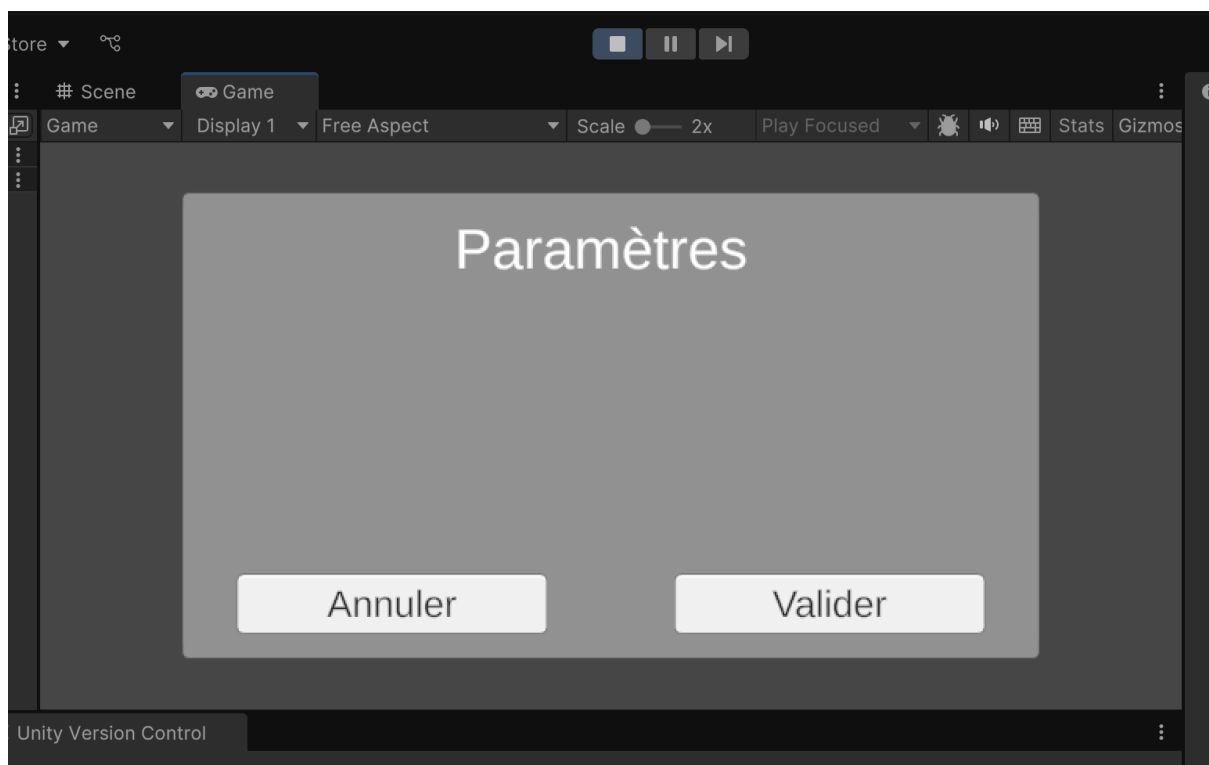
Quelques captures d'écrans de l'éditeur Unity



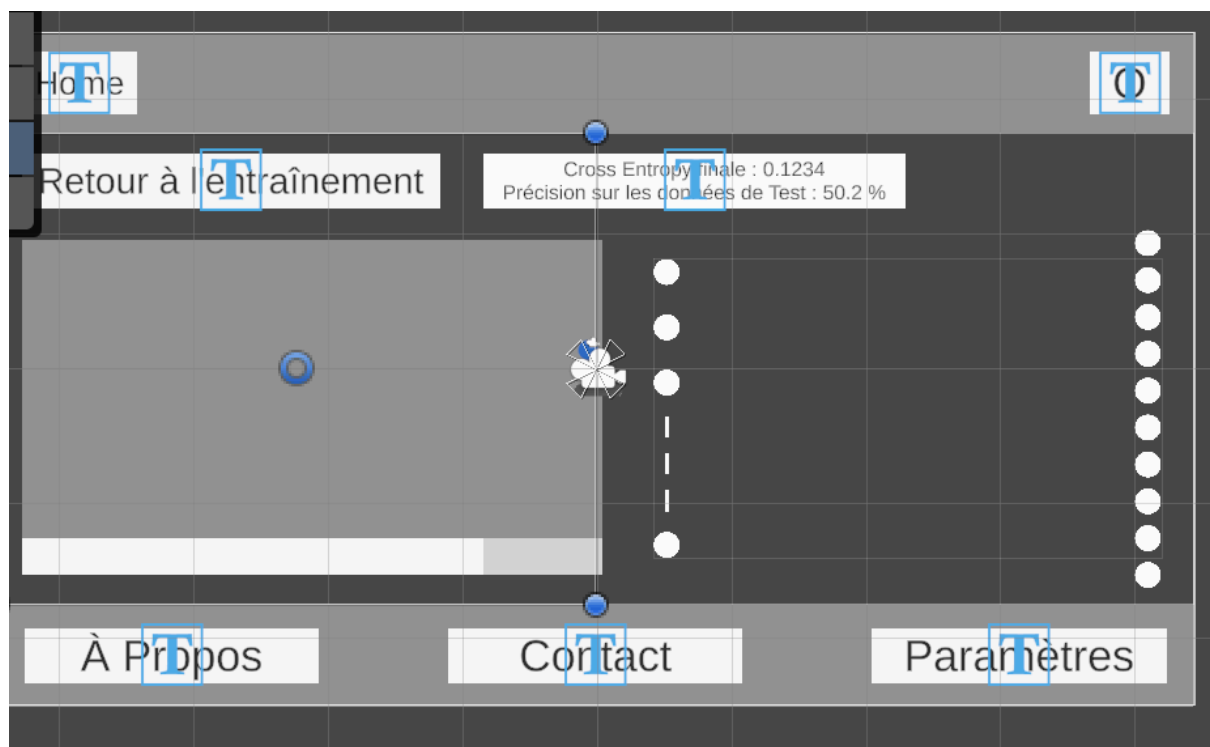
Onglet : À Propos



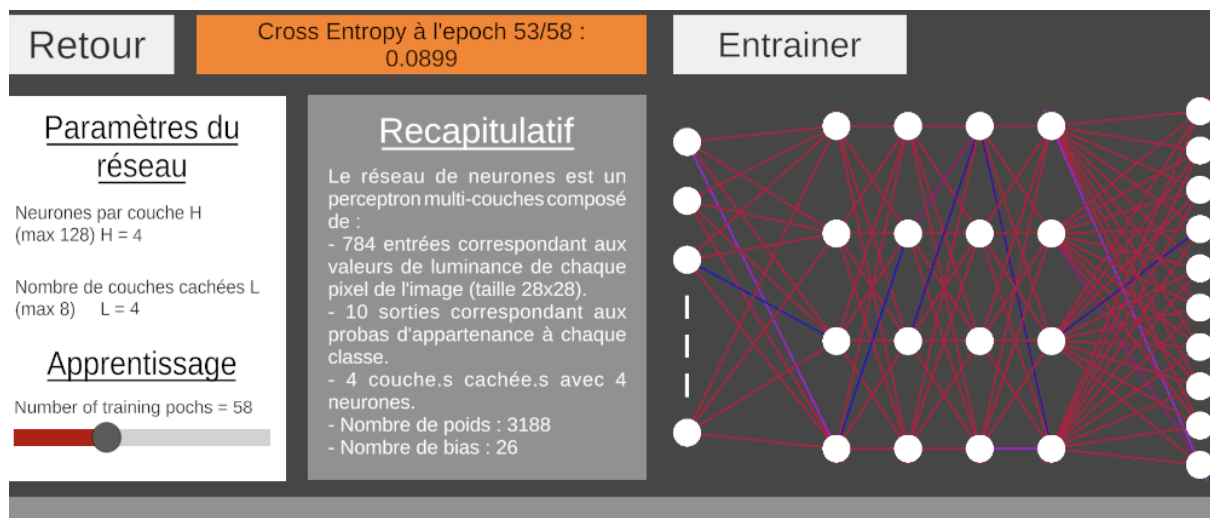
Onglet : Paramètres



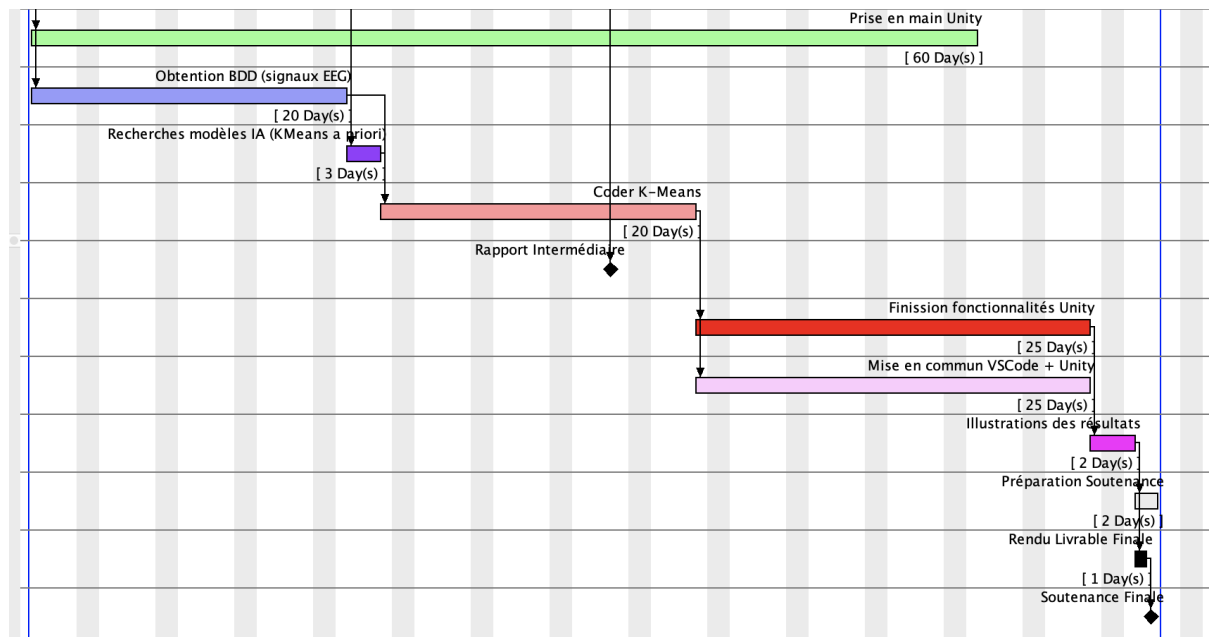
Fenêtre de test d'entraînement du MLP



Entraînement du MLP en cours



Planning final



Difficultés rencontrées

Les données

Le changement de sujet du projet au milieu de celui-ci a difficulté le planning parce que beaucoup du temps que j'avais dépensé à trouver le jeu de données, de le traiter (cf rapport intermédiaire), etc... a été soustrait du temps global du projet. Toutefois, c'était pas non plus du temps perdu, et c'était nécessaire pour l'avancement du projet et c'est grâce au changement des données d'études que j'ai pu enrichir le projet.

Unity

Une difficulté non prévue c'est plutôt technique et liée au matériel que j'avais à disposition. En effet, la prise en main d'Unity était une tâche qui a duré plus de temps que prévu, et c'est parce que entre plus de temps en passe dedans, plus on se rend compte des subtilités du logiciel, des limites, et surtout des capacités de rendu de mon ordinateur, qui m'a permis de passer beaucoup de temps à attendre que le projet charge, d'importer tous les assets, etc..

Exportation du projet

Même si c'est un projet qui a été développé entièrement en MacOS, exporter un exécutable pour Mac s'est avéré pas possible à cause des mesures de sécurité de l'OS. Heureusement que l'exécutable Windows a été un succès. Quelques bugs de dernière minute dans l'application finale ont aussi été difficiles à résoudre.

Perceptron multi-couches

J'aurais compris pourquoi on utilise souvent des Framework python pour créer des réseaux de neurones. Ce qu'on perd en puissance, on le gagne en compréhension et en facilité de coder les différentes fonctionnalités. Là où en C# Ça prend 150 lignes de codes, avec Python et numpy c'est fait en 10 lignes. Toutefois, je reste convaincu que je n'étais pas trop loin du succès.

Post Mortem

Liste des tâches qui restent à réaliser :

1. Implémenter complètement les fonctionnalités pour tester à la main l'entraînement du MLP et ne pas se contenter d'afficher sa précision.
2. Ajouter toutes ses fonctionnalités supplémentaires au projet Unity : des vrais paramètres réglables comme les couleurs, la transparence, taille des boutons, etc...
3. Ajout d'effets sonores et animations au réseau de neurones
4. Comprendre pourquoi les barycentres créés par K Means sont affichés à l'envers...
5. Soin esthétique des Gameobjects (ajouter des vrais images créés sur canva pour le fond, pour les boutons, etc...
6. Trouver comment faire pour générer correctement l'exécutable pour MacOS
7. Ajout de plus de personnalisation sur l'entraînement des modèles : choix du calcul de distance pour KMeans, choix du learning rate pour le MLP, ...